Guía de Scaffolding y Arquitectura

El **Scaffolding** (Andamiaje) es la práctica de crear físicamente la estructura de carpetas y archivos vacíos de un proyecto **antes** de escribir código o pedirle a la IA que programe.

> **¿Por qué esto controla a la IA?**

Si le pides a una IA que "cree un sistema de login" en un lienzo en blanco, la IA creará un único archivo gigante (Monolito). Si tú creas la estructura primero y le dices *"Rellena el archivo `/services/auth.ts`"*, estás obligando a la máquina a respetar el Principio de Responsabilidad Única.

1. Frontend (React / TypeScript)

La idea es aislar lo visual de la lógica de negocio.

```
/src
  ├── /assets          # Imágenes, iconos, fuentes.
  ├── /components      # Piezas visuales "Tontas" (Dumb components).
  │    ├── /ui         # Botones, inputs, modales.
  │    └── /forms      # Formularios específicos compuestos por UI.
  ├── /hooks           # Lógica de estado reutilizable (ej. useAuth).
  ├── /services        # Conexiones externas. ¡Única capa que habla con DB!
  ├── /store           # Estado global (ej. Zustand, Redux).
  ├── /utils           # Funciones puras (ej. formatearFecha).
  ├── /types           # Interfaces y tipos (Tus contratos).
  └── /pages           # Vistas que unen componentes, hooks y servicios.
```

¿Por qué se separa así?

- **Reutilización:** Un `<Button />` en `/ui` no tiene lógica de negocio.
- **Aislamiento de Errores:** Si la base de datos cambia, solo la carpeta `/services` se modifica.

2. Backend Serverless (Supabase / Deno)

En arquitecturas serverless, cada función es independiente pero comparten lógica.

```
/supabase
  ├── /functions           # El código de las Edge Functions
  │    ├── /auth-hook      # Función específica
  │    │    └── index.ts
  │    ├── /process-payment # Otra función aislada
  │    │    └── index.ts
  │    └── /_shared        # ¡CRÍTICO! Lógica compartida
  │         ├── cors.ts    # Cabeceras comunes
  │         ├── db.ts      # Cliente instanciado
  │         └── types.ts   # Tipos TypeScript comunes
```

¿Por qué se separa así? Al forzar una carpeta `/_shared`, obligas a la IA a importar las dependencias desde un solo lugar, previniendo que repita el código de conexión mil veces.

3. Zonas Libres de IA (El Dilema del Instrumento)

Aprender a programar es como aprender a tocar un instrumento. Si la IA siempre escribe tu código, sufrirás de "atrofia sintáctica".

1. **Diseño:** Crea las carpetas manualmente en tu terminal.
2. **Definición:** Crea los archivos `.ts` vacíos.
3. **Contratos:** Abre la carpeta `/types` y ESCRIBE FÍSICAMENTE las interfaces sin autocompletado. Esto formará tu criterio.
4. **Ejecución:** Ahora sí, da la orden a la IA: *"Lee los tipos en `/types/user.ts` y genérame el servicio."*

Conclusión

Si dominas este proceso, jamás sufrirás de deuda técnica. Eres un Arquitecto ensamblando módulos, no un programador escribiendo scripts gigantes.